

Cambodia Daily Consumer Price Index

Medallion Pipeline

Complete Technical Architecture, Econometric Foundations, Mathematical Derivations, and Systems Implementation Handbook

Publication Status: Production v2.4 (Definitive Master Edition)
Target Economy: Kingdom of Cambodia (Dollarized Developing Economy)
Governing International Standard: IMF/ILO/OECD/Eurostat (2020) CPI Manual
Official National Benchmark: NIS Cambodia (CSES Oct–Dec 2006 = 100.0)
Infrastructure Stack: Apache Airflow 2.9.3, dbt-core 1.8, PostgreSQL 16 Alpine

Author: Advanced Macroeconomic Engineering Team
Institution: National Inflation Intelligence Platform
Edition: Definitive Architectural & Mathematical Reference (September 2026)

Contents

1	Macroeconomic Context and Cambodian Inflation Dynamics	3
1.1	Structural Monetary Landscape: De Facto Dollarization	3
1.2	Extreme Consumer Expenditure Concentration	3
1.3	The Structural Deficits of Conventional Monthly Surveys	3
1.4	The High-Frequency Alternative	4
2	End-to-End System and Infrastructure Architecture	5
2.1	The Medallion Architectural Paradigm	5
2.2	Airflow DAG Execution Topology	5
3	Bronze Ingestion and Anti-Bot Web Engineering	7
3.1	Detailed Profile of 23 Scraper Channels	7
3.2	Anti-Bot Mitigation & Crawler Architecture	8
3.3	The Atomic Deduplication Contract	8
4	Silver Layer: Entity Resolution, Vector Embeddings and AI Classification	9
4.1	The Multilingual Script Disconnect in Cambodian Retail	9
4.2	Dense Multilingual Vector Space Architecture	9
4.3	High-Throughput BLAS Matrix Search & Top- K Introspect Partitioning	9
4.4	Deterministic Hardware & Packaging Specification Guards	10
4.5	Database-Native Vector Search via PostgreSQL <code>pgvector</code> and HNSW Graphs	10
4.6	The 3-Tier Zero-Crash Fallback Cascade	11
4.7	The 4-Tier UN COICOP Classification Ladder & Active Learning	11
5	Concrete Walkthrough: Life of a Price Quote	13
5.1	Stage 1: Raw Observation Ingestion (Bronze Layer)	13
5.2	Stage 2: Cleaning and Normalization (Silver Layer)	13
5.3	Stage 3: Entity Resolution and Classification	13
5.4	Stage 4: Elementary Price Index Compilation (Gold Layer)	13
5.5	Stage 5: Macroeconomic Laspeyres Aggregation	14
6	Econometric Price Index Theory and Axiomatic Proofs	15
6.1	The Axiomatic Approach to Index Numbers	15
6.2	Comparative Analysis of Elementary Index Formulas	15
7	Gold Layer: Econometric Compilation, Imputation and Hedonics	17
7.1	Compounded Class-Mean Geometric Imputation Engine	17
7.2	Time-Dummy Log-Linear Hedonic Quality Adjustment Engine	17
7.3	Macroeconomic Modified Laspeyres Rollup	17
8	Comprehensive Mathematical and Programmatic Equation Dictionary	19
8.1	Equation 1: Base Price Geometric Mean ($P_{i,0}$)	19
8.2	Equation 2: Individual Item Price Relative ($R_{i,t}$)	19
8.3	Equation 3: Class-Mean Movement Ratio ($R_{c,t}$)	19
8.4	Equation 4: Compounded Missing Price Imputation ($\hat{P}_{i,t}$)	20
8.5	Equation 5: Elementary Jevons Geometric Mean Index ($I_{J,c}^{0:t}$)	20
8.6	Equation 6: Subclass-Weighted Division Index ($I_{div,k}^{0:t}$)	21
8.7	Equation 7: Macroeconomic Modified Laspeyres Headline CPI ($CPI_{Headline}^{0:t}$)	21
8.8	Equation 8: Core CPI (Ex-Food & Energy) ($CPI_{Core}^{0:t}$)	21
8.9	Equation 9: Time-Dummy Log-Linear Hedonic Quality Regression	22

8.10	Equation 10: Nowcasting Leading Signal Drift ($\hat{\delta}_{\text{leading}}$)	22
8.11	Equation 11: Linear Trajectory Midpoint Expectation ($\mathbb{E}[\bar{P}_{\text{remaining}}]$)	22
8.12	Equation 12: Blended Month-to-Date Expected Index (Nowcast CPI_M)	23
8.13	Equation 13: Real-Time Splicing to Historical NIS Base ($\widehat{\text{CPI}}_{\text{NIS}, M}$)	23
8.14	Equation 14: Annual Continuous Series Chain-Linking Overlap Splice (S)	23
8.15	Quick Reference: Equations to Database Columns	24
9	Exhaustive UN COICOP Classification and Weights Hierarchy	25
10	High-Frequency Inflation Nowcasting and ML Ensemble	28
10.1	Mathematical Derivation of Linear Trajectory Drift	28
10.2	Automated Backtesting and Horizon Convergence Framework	28
10.3	Machine Learning Model Ensemble Architecture	29
11	Dual-Currency Econometrics and Exchange Rate Pass-Through	30
11.1	Modeling Exchange Rate Pass-Through (ERPT)	30
12	Chain-Linking to Official NIS Historical Benchmark	31
12.1	The Dual-Baseline Problem	31
12.2	Real-Time Nowcast Splicing	31
12.3	Annual Rebasing Overlap Splicing (December Overlap)	31
13	Data Lineage (dbt Core) and Automated Testing Suite	32
13.1	The Transformation Lineage	32
13.2	Structured Catalog of Automated Tests	32
13.3	Empirical Out-of-Sample Performance	32
14	Database Performance Tuning and PostgreSQL 16 Scaling	33
14.1	Storage Growth Projections	33
14.2	Index Optimization Strategy	33
14.3	Recommended PostgreSQL 16 Production Parameters	33
15	Executive Business Intelligence and Production SQL Catalog	34
15.1	Production SQL Query Catalog for Policy Dashboards	34
16	Strategic Policy Roadmap for the National Bank of Cambodia	35
16.1	High-Frequency Monetary Transmission Telemetry	35
16.2	Fiscal Applications for the Ministry of Economy and Finance (MEF)	35
16.3	Conclusion	35

1 Macroeconomic Context and Cambodian Inflation Dynamics

1.1 Structural Monetary Landscape: De Facto Dollarization

The Kingdom of Cambodia operates one of the most heavily dollarized financial architectures in the developing world. Emerging from geopolitical instability and economic reconstruction in the early 1990s, the United States Dollar (USD) established deep operational penetration alongside the national legal tender, the Cambodian Riel (KHR). Today, commercial bank deposits, large-scale consumer transactions, digital commerce pricing, and private-sector payrolls operate primarily in USD, while agricultural market transactions, public utility tariffs, civil service wages, and rural commerce operate primarily in KHR.

Under this dual-currency regime, consumer pricing behavior exhibits distinctive microeconomic characteristics:

1. **Dual-Currency Price Quotation:** Modern e-commerce portals, supermarket chains, and electronic retailers predominantly display prices in USD. Conversely, state-regulated utilities (such as Electricité du Cambodge and the Phnom Penh Water Supply Authority), passenger bus operators, and local open wet markets quote exclusively in KHR.
2. **Asymmetric Exchange Rate Pass-Through:** Fluctuations in the USD/KHR exchange rate (regulated through daily market operations by the National Bank of Cambodia) pass through directly into domestic prices with differing temporal speeds. Retailers quoting in USD pass exchange-rate volatility directly to KHR-denominated wage earners, while KHR-quoted staple foods respond to imported fertilizer and transport fuel costs.

1.2 Extreme Consumer Expenditure Concentration

According to the Cambodia Socio-Economic Survey (CSES) conducted by the National Institute of Statistics (NIS) under the Ministry of Planning, household spending is heavily concentrated in basic survival necessities:

- **Division 01 (Food and Non-Alcoholic Beverages): 44.775%** of national consumer expenditure.
- **Division 04 (Housing, Water, Electricity, Gas and Other Fuels): 17.084%** of national expenditure.
- **Division 07 (Transport and Automotive Fuels): 12.180%** of national expenditure.

These three consumption divisions account for **74.039%** of total household expenditure in Cambodia. In lower-income deciles, food expenditure alone exceeds 55% of monthly household budgets. Consequently, international commodity price shocks—such as spikes in global crude petroleum, regional supply disruptions in rice or pork across the Vietnamese and Thai borders, or climatic impacts on local fishing along the Tonle Sap—rapidly destabilize the macroeconomy.

1.3 The Structural Deficits of Conventional Monthly Surveys

Official inflation measurement compiled by the National Institute of Statistics (NIS) relies upon physical field surveys across urban and provincial markets. While strictly conforming to international manual protocols, field-collected surveys introduce structural operational limitations:

1. **The 20-to-30-Day Publication Lag Penalty:** Official monthly CPI bulletins are published 3 to 4 weeks after the close of the reference month. Under volatile macroeconomic shocks, central bankers at the National Bank of Cambodia (NBC) and fiscal planners at the Ministry of Economy and Finance (MEF) operate in an information deficit.

2. **Point-in-Time Mid-Month Sampling Bias:** Field enumerators typically audit physical retail stalls once per month (concentrated between the 10th and 15th calendar day). This static point sampling fails to capture dynamic intra-month price adjustments, promotional discounts, and supply shortages occurring late in the calendar month.
3. **Operational Overhead and Collection Fragility:** Field price collection requires hundreds of manual enumerators navigating traffic, physical store closures, and unstandardized weight estimates, introducing enumerator recording error.

1.4 The High-Frequency Alternative

The **Cambodia Daily CPI Medallion Pipeline** resolves these structural challenges by continuously ingesting over 35,500 daily price observations across 23 digital retail, utility, and telecom sources. The system compiles daily elementary and macroeconomic price indices conforming to IMF/ILO (2020) standards, providing real-time leading indicators and nowcasting official monthly benchmarks weeks ahead of government publication.

2 End-to-End System and Infrastructure Architecture

2.1 The Medallion Architectural Paradigm

To guarantee data lineage, auditability, and single-writer consistency, the platform implements the industrial **Medallion Data Architecture** across three structured layers:

Medallion Data Platform Flow

1. BRONZE LAYER (Raw Ingestion)

- 23 Daily Automated Web Crawlers & API Feeds (Supermarkets, Quick-Commerce, Pharmacies, Tech, Utilities, Transport).
- Ingested into table `bronze.raw_prices` with deterministic deduplication index `uq_raw_prices_observation`.
- Batch execution metadata stored in `staging.raw_scrapes` with JSONB payload concatenation on conflict.

↓ *Cleaning, Normalization, Entity Resolution & ML Classification*

2. SILVER LAYER (Harmonized Conformation)

- Multilingual text scrubbing (Khmer UTF-8 & English) and regex unit normalization (1000g = 1kg).
- Dual-currency conversion into Cambodian Riel (KHR) via daily official NBC/MEF exchange rates.
- Deterministic specification guardrails (storage capacity, pack quantities, mass/volume).
- Multi-tier entity resolution into canonical items (`silver.canonical_items`).
- 4-Tier UN COICOP classification ladder (Overrides → Pure Stores → Vector Centroids → Gemini LLM).
- Time-dummy log-linear hedonic quality adjustment engine (`silver.hedonic_adjusted_prices`).

↓ *Axiomatic Index Compilation, Imputation & Nowcasting*

3. GOLD LAYER (Analytical Metric Serving)

- Elementary Jevons geometric mean price index at 4-digit COICOP subclass level (`gold.fct_coicop_class_daily`).
- 7-day compounded class-mean geometric missing price imputation engine ($\hat{P}_{i,t} = P_{i,t-\Delta t} \cdot R_{c,t}^{\Delta t}$).
- Subclass-to-Division & Division-to-Headline Modified Laspeyres expenditure weight aggregation.
- Core CPI compilation (excluding Food 01, Housing/Utilities 04, Transport/Fuel 07).
- Real-time inflation nowcasting engine with linear trajectory drift expectation and dynamic 95% CI fan bands.
- Dual-chain linking to official historical NIS benchmark (Base: Oct–Dec 2006 = 100.0).

↓ *Executive Decision Support*

4. SERVING LAYER (Dashboards & Analytics): Metabase Portal and Power BI Strategic Suite.

2.2 Airflow DAG Execution Topology

The pipeline is orchestrated by Apache Airflow 2.9.3 running in Docker containers, executing three strictly ordered DAGs:

1. **Master Scraper DAG (`cpi_master_dag`):** Triggers daily at 02:00 Phnom Penh Time (UTC+7). Executes 23 scraper tasks concurrently across worker pools. Implements exponential backoff retries, checks output schemas, and asserts Bronze ingestion health before triggering downstream layers.
2. **Silver Conformation DAG (`silver_dag`):** Executes data cleaning, unit math normalization, currency conversion, deterministic specification guards, vector-based entity resolution, and the 4-tier UN COICOP classification ladder.

3. **Gold Econometric Compilation DAG (`gold_cpi_dag`):** Executes elementary Jevons price indexing, missing price imputation, hedonic quality regression, Laspeyres aggregation into Headline and Core CPI, nowcasting inference, and builds executive dimensional star-schemas.

3 Bronze Ingestion and Anti-Bot Web Engineering

3.1 Detailed Profile of 23 Scraper Channels

The platform captures over 35,500 raw daily quotes across 23 digital retail sources:

Data Source	Retail Segment	Extraction Technology	Tech-	COICOP Division
AEON Online Cambodia	Hypermarket Chain	Headless wright API	Play-	01, 02, 05, 11, 12
Lucky Supermarket	Supermarket Chain	Reverse Engineered REST	Engi-	01, 02, 05, 12
Chip Mong Supermarket	Premium Grocery	Mobile REST	Backend	01, 02, 05, 12
Makro Cambodia	Wholesale Warehouse	Mobile App Gateway	Gate-	01, 02, 05
GrabMart Lucky Feed	Quick Commerce	Session Gateway	Token	01, 02, 12
GrabMart Chip Mong	Quick Commerce	Session Gateway	Token	01, 02, 12
Bayon Supermarket	Local Supermarket	Structured Parser	HTML	01, 02, 05
Ucare Pharmacy	Modern Pharmacy	E-Commerce Catalog	Cata-	06 (Health), 12
GrabMart Ucare Feed	On-Demand Pharma	Session Gateway	Token	06 (Health)
Pharmacie de la Gare	Prescription Drugs	Web Scraper	Catalog	06 (Health)
Khmer24 Electronics	Tech Marketplace	DOM Traversal		08, 09 (Computing)
Nika Phone Shop	Consumer Tech	Structured data	Micro-	08 (Communication)
K-Store Electronics	IT & Computing	Product Feed	Detail	08, 09 (IT Hardware)
Sunsimexco Electronics	Consumer Appliances	E-Commerce Scraping		05 (Appliances)
Electricité du Cambodge	National Power Grid	Tariff Parser	Schedule	04.5.1 (Electricity)
PPWSA Water Supply	Municipal Water	Tariff Parser	Schedule	04.4.1 (Water)
PTT Station Cambodia	Retail Petroleum	Official Price Table		07.2.2 (Fuel)
Tela Cambodia	Petroleum Retailer	Daily Fuel Board	Fuel Price	07.2.2 (Fuel)
TotalEnergies Cambodia	Multinational Fuel	Fuel Board Ingestion		07.2.2 (Fuel)
Smart Axiata	Telecom & Broadband	Data Plan Catalog		08.2.0, 08.3.0
Cellcard Cambodia	Telecom & Broadband	Data Plan Catalog		08.2.0, 08.3.0
Zando Cambodia	Modern Apparel Chain	E-Commerce Scraping		03 (Apparel)
Pedro Cambodia	Footwear & Bags	DOM Extraction		03 (Footwear)

3.2 Anti-Bot Mitigation & Crawler Architecture

Commercial e-commerce portals deploy anti-bot systems (Cloudflare WAF, AWS Shield, DataDome). The pipeline implements three layers of crawler defense:

1. **TLS Fingerprint Mimicking (JA3/JA4 Spoofing):** Standard Python `requests` and `cURL` libraries are instantly flagged by modern WAFs due to rigid TLS cipher suites. The scrapers utilize `curl_cffi` and `Playwright` to impersonate standard Chrome 124 browser handshakes.
2. **Token Bucket Rate Limiting with Exponential Jitter:** To avoid HTTP 429 (Too Many Requests), requests are throttled with randomized exponential backoff:

$$T_{\text{wait}} = \min(T_{\text{max}}, T_{\text{base}} \times 2^{\text{attempt}}) \pm U(0, \text{jitter}) \quad (1)$$

3. **Session Token Reverse-Engineering:** For mobile gateways (GrabMart Lucky, GrabMart Chip Mong, GrabMart Ucare), authorization tokens are generated and refreshed using simulated mobile client headers.

3.3 The Atomic Deduplication Contract

To guarantee mathematical idempotency across Airflow task retries, `bronze.raw_prices` enforces a unique observation index:

```
CREATE UNIQUE INDEX IF NOT EXISTS uq_raw_prices_observation
ON bronze.raw_prices (
    store_id,
    source_name,
    COALESCE(source_url, ''),
    item_description_raw,
    ((scraped_at AT TIME ZONE 'UTC')::date)
);
```

The Price Invariance Principle: Nominal price is strictly excluded from this unique constraint. If an Airflow retry runs mid-day, the observation updates in-place via `ON CONFLICT DO UPDATE`, preventing duplicate quote accumulation.

4 Silver Layer: Entity Resolution, Vector Embeddings and AI Classification

4.1 The Multilingual Script Disconnect in Cambodian Retail

In Cambodian online retail marketplaces, product titles exhibit extreme linguistic heterogeneity. The exact same consumer SKU is listed across three divergent orthographic formats:

- 1. Khmer Script (Unicode range U+1780 to U+17FF):** e.g., "ក្របខ័ណ្ឌ 330ml".
- 2. Latin Script (English/French loan words):** e.g., "Angkor Beer 330ml Can".
- 3. Mixed-Script Code-Switching:** e.g., "ក្របខ័ណ្ឌ Angkor Premium Beer 330ml [Promo Pack]".

Standard string-distance metrics (Levenshtein distance, Jaro-Winkler, token-sort ratio) operate on character n-grams. When comparing Khmer script against Latin text, the set intersection of characters is empty ($\mathcal{C}_{\text{Khmer}} \cap \mathcal{C}_{\text{Latin}} = \emptyset$), yielding a similarity score of zero (0.000). Consequently, keyword heuristics fail completely across language boundaries.

4.2 Dense Multilingual Vector Space Architecture

To bridge the script divide, the pipeline projects all unstructured product strings into a continuous, dense 768-dimensional semantic embedding space $\mathbb{R}^{768}$ via transformer neural networks:

$$\mathbf{v} = \phi(\text{text}) \in \mathbb{R}^{768} \quad (2)$$

where $\phi(\cdot)$ denotes the embedding model (gemini-embedding-2 in cloud inference, or paraphrase-multilingual-MiniLM-L12-v2 in local CPU inference).

Every raw vector $\mathbf{v}$ is projected onto the unit hypersphere via L_2 -normalization:

$$\mathbf{u} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} = \frac{\mathbf{v}}{\sqrt{\sum_{d=1}^{768} v_d^2}}, \quad \text{such that } \|\mathbf{u}\|_2 = 1.0 \quad (3)$$

The semantic similarity between a candidate scraped listing $\mathbf{u}_{\text{cand}}$ and a canonical item $\mathbf{u}_{\text{base}}$ is evaluated via the geometric **Cosine Similarity**:

$$\mathcal{S}_{\cos}(\mathbf{u}_{\text{cand}}, \mathbf{u}_{\text{base}}) = \mathbf{u}_{\text{cand}} \cdot \mathbf{u}_{\text{base}} = \sum_{d=1}^{768} u_{\text{cand},d} u_{\text{base},d} \quad (4)$$

The corresponding **Cosine Distance** metric is defined as:

$$\mathcal{D}_{\text{cos}}(\mathbf{u}_{\text{cand}}, \mathbf{u}_{\text{base}}) = 1 - \mathcal{S}_{\text{cos}}(\mathbf{u}_{\text{cand}}, \mathbf{u}_{\text{base}}) \quad (5)$$

Because the vector representation maps semantic intent rather than literal spelling, " $\square\square\square\square\square\square\square\square\square\square$ " and "*Angkor Beer*" map to proximate coordinate clusters on the 768-dimensional manifold, achieving cosine similarities exceeding $\mathcal{S}_{\text{cos}} > 0.93$.

4.3 High-Throughput BLAS Matrix Search & Top- K Introselect Partitioning

During daily ingestion batches of 35,500 observations, computing individual dot products sequentially in Python loops would impose prohibitive computational latency ($\mathcal{O}(M \times N)$). The pipeline implements vectorized BLAS matrix multiplication:

$$\mathbf{S} = \mathbf{M} \cdot \mathbf{u}_{\text{cand}} \in \mathbb{R}^N \quad (6)$$

where $\mathbf{M} \in \mathbb{R}^{N \times 768}$ is the contiguous pre-stacked memory matrix of all N active canonical item unit vectors cached in RAM.

To eliminate the $\mathcal{O}(N \log N)$ cost of fully sorting the similarity vector $\mathbf{S}$ across tens of thousands of items, the engine executes **Top- K Introspect Partitioning** via `numpy.argpartition`:

$$\mathcal{K}_{\text{top}} = \text{argpartition}(\mathbf{S}, -30)[-30 :] \quad (7)$$

Introspect isolates the 30 nearest semantic neighbors in linear time $\mathcal{O}(N)$. Full sorting is subsequently performed only over this constrained 30-element candidate subset in $\mathcal{O}(K \log K)$ with $K = 30$, reducing neighbor discovery latency to under 1.5 milliseconds.

4.4 Deterministic Hardware & Packaging Specification Guards

Pure vector semantic matching presents a fatal vulnerability for inflation measurement: high-dimensional neural models cluster related goods closely. For example, "*Coca-Cola 330ml Can*" and "*Coca-Cola 330ml Pack of 24*" exhibit semantic similarity $\mathcal{S}_{\text{cos}} \approx 0.88$. Merging them would inject an artificial 2,300% price shock into the elementary Jevons index.

To prevent false positive merges, the pipeline establishes **Deterministic Specification Guards** (`is_spec_compatible`) that override vector similarity:

1. **Electronics Storage Invariant:** For technology listings (COICOP Division 08/09), flash storage capacities must match exactly:

$$\text{StorageGuard} = \mathbf{1}_{\{S_{\text{cand}}=S_{\text{base}} \vee S_{\text{cand}}=\emptyset \vee S_{\text{base}}=\emptyset\}} \quad (8)$$

A 128GB iPhone and a 256GB iPhone are permanently prohibited from matching.

2. **Packaging Quantity Multiplier Guard:** Multi-pack quantities extracted via regular expressions ($Q_{\text{pack}} \in \{1, 6, 12, 24, 48\}$) must be identical:

$$\text{PackGuard} = \mathbf{1}_{\{Q_{\text{cand}}=Q_{\text{base}}\}} \quad (9)$$

A single can ($Q = 1$) is never merged with a 6-pack ($Q = 6$).

3. **Normalized Physical Volume/Mass Metric Tolerance:** Product masses (g, kg) and volumes (ml, L) are cross-normalized to base SI units ($1000g = 1kg$, $1000ml = 1L$). Merging is rejected if volume discrepancy exceeds 10%:

$$\text{VolumeGuard} = \mathbf{1}_{\left\{ \frac{|V_{\text{cand}} - V_{\text{base}}|}{\max(V_{\text{cand}}, V_{\text{base}})} \leq 0.10 \right\}} \quad (10)$$

4. **Dietary & Formulation Disconnect:** Products with formulation keywords ("*Zero*", "*Diet*", "*Light*", "*No Sugar*") cannot merge with standard sugar-sweetened counterparts.

4.5 Database-Native Vector Search via PostgreSQL `pgvector` and HNSW Graphs

To eliminate the memory footprint of holding large catalog matrices in Python worker RAM, the production database implements native vector storage via the PostgreSQL `pgvector` extension.

1. Schema DDL & HNSW Index Definition

Canonical product records in `silver.canonical_items` are augmented with 768-dimensional native vector columns:

```
CREATE EXTENSION IF NOT EXISTS vector;

ALTER TABLE silver.canonical_items
ADD COLUMN IF NOT EXISTS embedding vector(768);

-- Hierarchical Navigable Small World (HNSW) graph index
CREATE INDEX IF NOT EXISTS idx_canonical_items_hnsw
ON silver.canonical_items USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

2. The HNSW Graph Traversal Algorithm

The HNSW index constructs a multi-layer graph where lower layers contain all data points with dense local clustering, and upper layers contain sparse long-range skip links. Query execution achieves logarithmic search complexity $\mathcal{O}(\log N)$:

- $M = 16$: Maximum number of bidirectional connection links per node in the proximity graph.
- $ef_construction = 64$: Size of the dynamic candidate list evaluated during index construction, balancing build time and recall precision.

3. Sub-Millisecond Cosine Nearest-Neighbor Query

Using the PostgreSQL cosine distance operator ($\leq$), the database executes vector similarity queries directly within the query planner:

```
SELECT
    item_id,
    canonical_name,
    brand,
    size_norm,
    coicop_division,
    coicop_code,
    1 - (embedding <=> %s::vector) AS cosine_similarity
FROM silver.canonical_items
WHERE embedding IS NOT NULL
ORDER BY embedding <=> %s::vector
LIMIT 30;
```

4.6 The 3-Tier Zero-Crash Fallback Cascade

To ensure 24/7 continuous operation without downtime during cloud API outages or rate limit exhaustion (HTTP 429), `VectorItemMatcher.embed_text()` executes a 3-tier cascade:

1. **Tier A (Cloud LLM Vector Engine):** Google `gemini-embedding-2` (768-dim) with 3-key round-robin load balancing via `GeminiKeyPool`.
2. **Tier B (Local High-Speed Neural Fallback):** `paraphrase-multilingual-MiniLM-L12-v2` executed on local CPU/GPU tensors. Vectors are dynamically padded and projected to standard 768 dimensions.
3. **Tier C (Deterministic Token Hashing Fallback):** In offline or isolated test environments, an internal Khmer-English synonym dictionary (40+ Cambodian retail pairs) is tokenized and projected into an L_2 -normalized 768-dimensional float32 vector using modulo hashing.

4.7 The 4-Tier UN COICOP Classification Ladder & Active Learning

Every canonical item is categorized into the 12-division UN COICOP hierarchy via:

1. **Tier 1 (Deterministic Overrides):** Curated regulatory and brand overrides in `silver.coicop_override`.

2. **Tier 2 (Single-Category Domain Purity):** 15 pure single-category stores are mapped instantaneously in SQL (EDC $\rightarrow$ 04.5.1, Tela/PTT $\rightarrow$ 07.2.2, Smart/Cellcard $\rightarrow$ 08.2.0, PPWSA $\rightarrow$ 04.4.1, BookMeBus $\rightarrow$ 07.3.2).
3. **Tier 3 (Centroid Vector Projection):** For multi-category supermarket catalogs (AEON, Lucky, Chip Mong), item vectors are compared against pre-computed bilingual UN COICOP division centroids $\mathbf{C}_k \in \mathbb{R}^{768}$. Matches with $S_{\cos} \geq 0.72$ are assigned automatically.
4. **Tier 4 (Active Learning Centroid Updating):** When new items are verified with high confidence (Score ≥ 0.95), division centroids update dynamically via an Exponential Moving Average (EMA):

$$\mathbf{C}_k^{(t)} = (1 - \alpha)\mathbf{C}_k^{(t-1)} + \alpha\mathbf{u}_{\text{new}}, \quad \text{with } \alpha = 0.02 \quad (11)$$

allowing the semantic vector space to continuously absorb emerging Cambodian brand names and colloquial packaging formats.

5 Concrete Walkthrough: Life of a Price Quote

To understand how microdata traverses the pipeline, consider a single real-world price observation: a 330ml can of Coca-Cola sold at Lucky Supermarket.

5.1 Stage 1: Raw Observation Ingestion (Bronze Layer)

At 02:14 AM Phnom Penh time, the Lucky Supermarket crawler scrapes the beverage section. The item is captured in raw JSON format:

```
INSERT INTO bronze.raw_prices (
  store_id, source_name, source_url, item_description_raw, price, currency, scraped_at
) VALUES (
  'lucky_supermarket', 'lucky_scraper',
  'https://lucky.com.kh/product/coca-cola-can-330ml',
  'Coca Cola Can 330ml [Special Offer]', 0.65, 'USD', '2026-09-06 02:14:00+07'
) ON CONFLICT (store_id, source_name, COALESCE(source_url, ''), item_description_raw, ((scraped_at AT TIME
  ZONE 'UTC')::date))
DO UPDATE SET price = EXCLUDED.price;
```

5.2 Stage 2: Cleaning and Normalization (Silver Layer)

During the 03:00 AM Silver DAG run, the raw record is processed:

1. **Text Standardization:** The title is cleaned of promotional tags: 'Coca Cola Can 330ml [Special Offer]' → 'Coca Cola Can 330ml'.
2. **Unit Extraction:** The regex parser identifies volume: `size_value = 330`, `size_unit = 'ml'`, `pack_qty = 1`.
3. **Currency Conversion:** The price of \$0.65 USD is converted to KHR using the official daily NBC exchange rate (4,100 KHR/USD):

$$P_{\text{KHR}} = 0.65 \times 4,100 = 2,665.00 \text{ KHR} \quad (12)$$

4. **Unit Price Normalization:** Converted to standard price per liter:

$$P_{\text{Unit, KHR}} = \frac{2,665.00}{0.330} = 8,075.76 \text{ KHR/L} \quad (13)$$

5.3 Stage 3: Entity Resolution and Classification

The item is matched against `silver.canonical_items`:

- Barcode: 8851959132014 matches an existing canonical item ID: 'e7b1a2c4-5d8f-4e9a-9b1c-3f2e1a0b5c4d'.
- Specification Guard: Single-can volume (330ml) is verified against the canonical definition (not a 24-can crate).
- Classification: Matched into UN COICOP Division **01** (Food & Non-Alcoholic Beverages), Group **01.2** (Non-Alcoholic Beverages), Class **01.2.2** (Mineral waters, soft drinks, juices).

5.4 Stage 4: Elementary Price Index Compilation (Gold Layer)

On the base date ($t = 0$), the geometric mean price of this item was 2,500.00 KHR. Today's price is 2,665.00 KHR. The individual price relative is:

$$R_{i,t} = \frac{2,665.00}{2,500.00} = 1.0660 \quad (+6.60\% \text{ relative to base}) \quad (14)$$

Within Class 01.2.2 (*Soft drinks and juices*), there are 45 observed beverage varieties. The elementary Jevons index is compiled as:

$$I_{J,01.2.2}^{0:t} = \left(\prod_{j=1}^{45} \frac{P_{j,t}}{P_{j,0}} \right)^{1/45} \times 100.0 = 104.25 \quad (15)$$

5.5 Stage 5: Macroeconomic Laspeyres Aggregation

Class 01.2.2 has an expenditure weight of $w_{01.2.2} = 2.280\%$. It aggregates into Division 01 (*Food*, weight 44.775%). Division 01 aggregates into national Headline CPI:

$$\text{CPI}_{\text{Headline}}^{0:t} = \sum_{k=1}^{12} W_k \cdot I_{\text{div},k}^{0:t} = 103.82 \quad (16)$$

Thus, our single can of Coca-Cola flows into the official daily inflation figure.

6 Econometric Price Index Theory and Axiomatic Proofs

6.1 The Axiomatic Approach to Index Numbers

A price index $I(P_0, P_t)$ aggregates vector prices from base period 0 to target period t . In international index number theory (Diewert, 1995; IMF/ILO, 2020), candidate index formulas are evaluated against core axiomatic properties:

Axiom 1 (Proportionality Axiom). *If all prices in period t increase by a constant factor $\lambda > 0$, the index must increase by exactly λ : $I(P_0, \lambda P_0) = \lambda$.*

Axiom 2 (Time Reversal Test). *An index satisfies the Time Reversal Test if reversing the base and comparison periods yields the reciprocal index:*

$$I(P_0, P_t) \times I(P_t, P_0) = 1 \quad (17)$$

Axiom 3 (Circularity and Transitivity Axiom). *An index satisfies Transitivity if a multi-period comparison equals the product of chained intermediate comparisons:*

$$I(P_0, P_t) = I(P_0, P_1) \times I(P_1, P_2) \times \cdots \times I(P_{t-1}, P_t) \quad (18)$$

Axiom 4 (Commensurability / Dimensional Invariance Test). *An index must be invariant to changes in the units of measurement for commodities.*

6.2 Comparative Analysis of Elementary Index Formulas

At the elementary aggregate level, expenditure quantities $Q_{i,t}$ are unavailable in real-time web-scraped data. National statistical offices choose between three unweighted formulas:

1. **Carli Index (Arithmetic Mean of Relatives):** $I_C = \frac{1}{n} \sum_{i=1}^n \left(\frac{P_{i,t}}{P_{i,0}} \right)$
2. **Dutot Index (Ratio of Arithmetic Averages):** $I_D = \frac{\frac{1}{n} \sum P_{i,t}}{\frac{1}{n} \sum P_{i,0}}$
3. **Jevons Index (Geometric Mean of Relatives):** $I_J = \prod_{i=1}^n \left(\frac{P_{i,t}}{P_{i,0}} \right)^{1/n}$

Table 2: Axiomatic Evaluation of Elementary Aggregate Formulas

Axiomatic Test	Carli (I_C)	Dutot (I_D)	Jevons (I_J)
Proportionality Axiom	Satisfied	Satisfied	Satisfied
Time Reversal Test	Violated	Satisfied	Satisfied
Circularity / Transitivity	Violated	Satisfied	Satisfied
Commensurability (Unit Invariance)	Satisfied	Violated	Satisfied
Monotonicity	Satisfied	Satisfied	Satisfied
Formula Bias	Severe Upward	Sample-Dependent	Zero Bias

Theorem 6.1 (Failure of the Carli Index and Upward Formula Drift). *The Carli arithmetic index violates the Time Reversal Test and causes severe upward formula bias due to Jensen's Inequality:*

$$I_C(P_0, P_t) \times I_C(P_t, P_0) \geq 1 \quad (19)$$

Equality holds strictly if and only if all individual price relatives are identical. In online retail where daily prices fluctuate around a constant mean (price bouncing), the Carli index creates substantial artificial inflation drift.

Proof. Let $R_i = \frac{P_{i,t}}{P_{i,0}}$. The forward Carli index is $I_C(P_0, P_t) = \frac{1}{n} \sum R_i$. The backward Carli index is $I_C(P_t, P_0) = \frac{1}{n} \sum \frac{1}{R_i}$. By the Cauchy-Schwarz inequality (or Jensen's inequality applied to the strictly convex function $f(x) = 1/x$ for $x > 0$):

$$\left(\frac{1}{n} \sum_{i=1}^n R_i \right) \left(\frac{1}{n} \sum_{i=1}^n \frac{1}{R_i} \right) \geq 1 \quad (20)$$

Strict inequality holds whenever variance across price relatives exists. Thus, I_C drifts upward over time. $\square$

Theorem 6.2 (Axiomatic Superiority of the Jevons Index). *The Jevons unweighted geometric mean price index satisfies the Time Reversal Test, Transitivity Axiom, and Commensurability Test, completely eliminating elementary formula bias.*

Proof. Evaluating time reversal:

$$I_J(P_t, P_0) = \prod_{i=1}^n \left(\frac{P_{i,0}}{P_{i,t}} \right)^{1/n} = \left[\prod_{i=1}^n \left(\frac{P_{i,t}}{P_{i,0}} \right)^{1/n} \right]^{-1} = \frac{1}{I_J(P_0, P_t)} \quad (21)$$

Evaluating transitivity:

$$I_J(P_0, P_1) \times I_J(P_1, P_2) = \frac{\prod P_{i,1}^{1/n}}{\prod P_{i,0}^{1/n}} \times \frac{\prod P_{i,2}^{1/n}}{\prod P_{i,1}^{1/n}} = \frac{\prod P_{i,2}^{1/n}}{\prod P_{i,0}^{1/n}} = I_J(P_0, P_2) \quad (22)$$

Both tests hold identically across all periods. $\square$

7 Gold Layer: Econometric Compilation, Imputation and Hedonics

7.1 Compounded Class-Mean Geometric Imputation Engine

In high-frequency online price monitoring, products frequently experience stockouts. International statistical standards (IMF/ILO, 2020) strictly prohibit flat carry-forward ($\hat{P}_{i,t} = P_{i,t-1}$) as static carry-forward artificially dampens true price volatility and creates downward lag bias during inflationary cycles.

When item i in division c is unobserved on day t with an elapsed gap $\Delta t \in [1, 7]$ days:

1. Compute the 1-day geometric mean movement ratio of all observed items in division c :

$$R_{c,t} = \exp \left(\frac{1}{|M_{c,t}|} \sum_{j \in M_{c,t}} \ln \left(\frac{P_{j,t}}{P_{j,t-1}} \right) \right), \quad R_{c,t} \in [0.80, 1.25] \quad (23)$$

2. Compound the daily movement over the actual elapsed gap Δt :

$$\hat{P}_{i,t} = P_{i,t-\Delta t} \times (R_{c,t})^{\Delta t} \quad (24)$$

3. If an item remains unobserved for $\Delta t > 7$ days, it is excluded from the active basket (churn exclusion).

7.2 Time-Dummy Log-Linear Hedonic Quality Adjustment Engine

In consumer electronics (Division 08), rapid technological turnover causes older models to be replaced by upgraded hardware at higher nominal prices. Raw price comparisons conflate general inflation with quality improvements (e.g., higher RAM, increased storage). The hedonic regression engine in `pipeline/hedonic_regress` estimates:

$$\ln P_{i,t} = \alpha + \sum_{k=1}^K \beta_k z_{i,k} + \sum_{\tau=1}^T \delta_\tau D_{i,\tau} + \epsilon_{i,t} \quad (25)$$

Table 3: Hedonic Quality Adjustment Regression Estimates for Electronics

Variable / Characteristic	Coefficient ($\hat{\beta}$)	Std. Error	t-Statistic	p-Value
Intercept (α)	12.450	0.082	151.8	< 0.001
ln(Storage GB)	0.412	0.024	17.16	< 0.001
ln(RAM GB)	0.285	0.031	9.19	< 0.001
Apple Brand Premium	0.534	0.045	11.86	< 0.001
Samsung Brand Premium	0.312	0.042	7.42	< 0.001
Xiaomi Brand Dummy	0.118	0.038	3.11	0.002

The model yields $R^2 = 0.874$. When design matrices exhibit rank-deficiency, the engine flags `SKIPPED_RANK_DEFICIENT` and safely defaults to matched-model pricing without pipeline interruption.

7.3 Macroeconomic Modified Laspeyres Rollup

Subclasses are aggregated into 2-digit divisions using official subclass weights:

$$I_{\text{div},k}^{0:t} = \frac{\sum_{c \in \text{div}_k} w_c \cdot I_{J,c}^{0:t}}{\sum_{c \in \text{div}_k} w_c} \quad (26)$$

Division indices are aggregated into national ****Headline CPI**** using CSES national expenditure weights:

$$\text{CPI}_{\text{Headline}}^{0:t} = \frac{\sum_{k=1}^{12} W_k \cdot I_{\text{div},k}^{0:t}}{\sum_{k=1}^{12} W_k \cdot \mathbf{1}_{[\text{active}_k]}} \quad (27)$$

Active-Weight Normalization: If a division has zero observed items on a given day, its weight is excluded from both numerator and denominator, preventing synthetic deflationary drag toward zero.

Core CPI (Ex-Food and Energy): In alignment with the National Bank of Cambodia and NIS, Core CPI strips out Division 01 (Food), Division 04 (Housing/Utilities), and Division 07 (Transport/Fuel):

$$\text{CPI}_{\text{Core}}^{0:t} = \frac{\sum_{k \notin \{01,04,07\}} W_k \cdot I_{\text{div},k}^{0:t}}{\sum_{k \notin \{01,04,07\}} W_k} \quad (28)$$

8 Comprehensive Mathematical and Programmatic Equation Dictionary

This chapter provides an exhaustive dictionary of all fourteen mathematical formulas powering the Cambodia Daily CPI Medallion Pipeline. For each equation, we detail its formal economic derivation, variable definitions, implementation in Python and SQL, and database destination.

8.1 Equation 1: Base Price Geometric Mean ($P_{i,0}$)

$$P_{i,0} = \exp \left(\frac{1}{|S_{i,0}|} \sum_{s \in S_{i,0}} \ln P_{i,0,s} \right) = \left(\prod_{s \in S_{i,0}} P_{i,0,s} \right)^{1/|S_{i,0}|} \quad (29)$$

- **Economic Purpose:** Computes an unweighted geometric mean baseline price for item i across all reporting stores on the base date ($t = 0$), eliminating store-selection bias.
- **Variables:**
 - i : Unique canonical item identifier (`item_id`).
 - 0: Base period reference date (e.g., earliest valid scrape date).
 - $P_{i,0}$: Baseline reference price of item i in Cambodian Riel (KHR).
 - s : Retail store or data provider ($s \in S_{i,0}$).
 - $|S_{i,0}|$: Total count of distinct stores quoting item i on the base date.
 - $P_{i,0,s}$: Observed unit price of item i at store s on the base date.

- **Code Implementation:** `pipeline/cpi_calculator.py` → `compute_base_prices()`:

```
grouped = base_df.groupby("item_id").agg(
    base_price_khr=("unit_price_khr", lambda x: float(np.exp(np.mean(np.log(x[x > 0])))))
)
```

- **Database Persistence:** `gold.fct_elementary_indices.base_price_khr`.

8.2 Equation 2: Individual Item Price Relative ($R_{i,t}$)

$$R_{i,t} = \frac{P_{i,t}}{P_{i,0}} \quad (30)$$

- **Economic Purpose:** Quantifies the price trajectory of a single homogeneous product on day t relative to its baseline period.
- **Variables:**
 - $R_{i,t}$: Price ratio of product i on day t .
 - $P_{i,t}$: Realized or imputed price of product i on day t (KHR).
 - $P_{i,0}$: Baseline reference price from Equation 1 (KHR).

- **Code Implementation:** `pipeline/cpi_calculator.py` → `compute_daily_elementary_indices()`:

```
valid["price_ratio"] = valid["current_price_khr"] / valid["base_price_khr"]
valid["price_ratio_pct"] = valid["price_ratio"] * 100.0
```

- **Database Persistence:** `gold.fct_elementary_indices.price_ratio`.

8.3 Equation 3: Class-Mean Movement Ratio ($R_{c,t}$)

$$R_{c,t} = \exp \left(\frac{1}{|M_{c,t}|} \sum_{j \in M_{c,t}} \ln \left(\frac{P_{j,t}}{P_{j,t-1}} \right) \right) = \left(\prod_{j \in M_{c,t}} \frac{P_{j,t}}{P_{j,t-1}} \right)^{1/|M_{c,t}|} \quad (31)$$

- **Economic Purpose:** Measures the daily geometric average rate of price change across observed products within COICOP division c between day $t - 1$ and day t .

- **Variables:**

- c : 2-digit COICOP division ($c \in \{01, 02, \dots, 12\}$).
- $R_{c,t}$: 1-day geometric mean price movement ratio for category c , clamped to $[0.80, 1.25]$.
- j : A matched product belonging to division c .
- $M_{c,t}$: Set of matched products in division c observed on both day t and day $t - 1$.
- $|M_{c,t}|$: Total number of matched observations.
- $\frac{P_{j,t}}{P_{j,t-1}}$: 1-day price relative for item j .

- **Code Implementation:** `pipeline/cpi_calculator.py` → `compute_daily_elementary_indices()`:

```
common["ratio"] = common["current_price_khr"] / common["unit_price_khr"]
for div, group in common.groupby("coicop_division"):
    division_movement_ratios[str(div)] = float(np.exp(np.mean(np.log(group["ratio"]))))
```

8.4 Equation 4: Compounded Missing Price Imputation ($\hat{P}_{i,t}$)

$$\hat{P}_{i,t} = P_{i,t-\Delta t} \times (R_{c,t})^{\Delta t} \quad \text{for } \Delta t \in [1, 7] \quad (32)$$

- **Economic Purpose:** Imputes prices for temporarily out-of-stock items by compounding the category growth rate over the elapsed missing window, avoiding static carry-forward lag bias.

- **Variables:**

- $\hat{P}_{i,t}$: Imputed unit price for item i on day t (KHR).
- Δt : Elapsed days since item i was last observed ($1 \leq \Delta t \leq 7$).
- $P_{i,t-\Delta t}$: Most recent observed price of item i .
- $R_{c,t}$: Category daily movement ratio from Equation 3.

- **Code Implementation:** `pipeline/cpi_calculator.py` → `compute_daily_elementary_indices()`:

```
days_gap = max(1, min((pd.to_datetime(calc_date) - pd.to_datetime(last_obs_date)).days, 7))
imputed_price = float(val) * (movement ** days_gap)
merged.at[idx, "current_price_khr"] = imputed_price
merged.at[idx, "is_imputed"] = True
```

- **Database Persistence:** `gold.fct_elementary_indices.current_price_khr (is_imputed = TRUE)`.

8.5 Equation 5: Elementary Jevons Geometric Mean Index ($I_{J,c}^{0:t}$)

$$I_{J,c}^{0:t} = \left(\prod_{i=1}^{n_c} \frac{P_{i,t}}{P_{i,0}} \right)^{1/n_c} \times 100.0 = \exp \left(\frac{1}{n_c} \sum_{i=1}^{n_c} \ln \left(\frac{P_{i,t}}{P_{i,0}} \right) \right) \times 100.0 \quad (33)$$

- **Economic Purpose:** Axiomatically robust elementary price index at the 4-digit/5-digit subclass level, satisfying time-reversal and transitivity tests.

- **Variables:**

- $I_{J,c}^{0:t}$: Jevons elementary index for subclass c on day t (base = 100.0).
- n_c : Number of active goods in subclass c .
- $P_{i,t}$: Price on day t (realized or imputed).
- $P_{i,0}$: Base price.

- **Code Implementation:** `pipeline/cpi_calculator.py` → `compute_jevons_index()`:

```
ratios = cur / base
return float(np.exp(np.mean(np.log(ratios))) * 100.0)
```

- **Database Persistence:** `gold.fct_coicop_class_daily.elementary_index`.

8.6 Equation 6: Subclass-Weighted Division Index ($I_{\text{div},k}^{0:t}$)

$$I_{\text{div},k}^{0:t} = \frac{\sum_{c \in \text{div}_k} w_c \cdot I_{J,c}^{0:t}}{\sum_{c \in \text{div}_k} w_c} \quad (34)$$

- **Economic Purpose:** Aggregates 4-digit subclass indices into 2-digit COICOP division indices using official CSES subclass weights.
- **Variables:**
 - k : COICOP division code (01 to 12).
 - $I_{\text{div},k}^{0:t}$: Aggregated price index for division k on day t .
 - w_c : Subclass expenditure weight (e.g., 17.23% for Bread and Cereals).
 - $I_{J,c}^{0:t}$: Elementary Jevons index for subclass c .
- **Code Implementation:** `pipeline/cpi_calculator.py` → `aggregate_division_and_headline()`:


```
div_index = sum(idx * wt for idx, wt in zip(subclass_indices, subclass_wts)) / sum(subclass_wts)
```
- **Database Persistence:** `gold.fct_cpi_daily.division_index`.

8.7 Equation 7: Macroeconomic Modified Laspeyres Headline CPI ($\text{CPI}_{\text{Headline}}^{0:t}$)

$$\text{CPI}_{\text{Headline}}^{0:t} = \frac{\sum_{k=1}^{12} W_k \cdot I_{\text{div},k}^{0:t}}{\sum_{k=1}^{12} W_k \cdot \mathbf{1}_{[\text{active}_k]}} \quad (35)$$

- **Economic Purpose:** Produces national headline inflation by weighting all 12 division indices by their national expenditure shares, with active-weight normalization.
- **Variables:**
 - $\text{CPI}_{\text{Headline}}^{0:t}$: National headline consumer price index on day t (base = 100.0).
 - W_k : National CSES expenditure weight for division k ($\sum W_k = 1.0$).
 - $\mathbf{1}_{[\text{active}_k]}$: Binary indicator (1 if division k has observations, 0 if empty).
- **Code Implementation:** `pipeline/cpi_calculator.py` → `aggregate_division_and_headline()`:


```
active_div = df_div[df_div["item_count"] > 0]
total_weight = active_div["weight"].sum()
headline_cpi = float((active_div["weight"] * active_div["division_index"]).sum() / total_weight)
```
- **Database Persistence:** `gold.fct_cpi_daily.headline_cpi` and `gold.fct_cpi_monthly.monthly_headline_cpi`.

8.8 Equation 8: Core CPI (Ex-Food & Energy) ($\text{CPI}_{\text{Core}}^{0:t}$)

$$\text{CPI}_{\text{Core}}^{0:t} = \frac{\sum_{k \notin \{01,04,07\}} W_k \cdot I_{\text{div},k}^{0:t}}{\sum_{k \notin \{01,04,07\}} W_k} \quad (36)$$

- **Economic Purpose:** Measures underlying structural inflation by removing volatile Food (01), Housing/Utilities (04), and Transport/Fuel (07). Denominator is renormalized to 25.961%.
- **Variables:**
 - $\text{CPI}_{\text{Core}}^{0:t}$: Core price index level on day t .
 - $k \notin \{01, 04, 07\}$: Condition selecting Divisions 02, 03, 05, 06, 08, 09, 10, 11, 12.
 - $\sum W_k$: Total core basket weight (0.25961).
- **Code Implementation:** `pipeline/cpi_calculator.py` → `aggregate_division_and_headline()`:


```
core_exclusions = {"01", "04", "07"}
core_divisions = active_div[~active_div["coicop_division"].isin(core_exclusions)]
core_cpi = float((core_divisions["weight"] * core_divisions["division_index"]).sum() /
                  core_divisions["weight"].sum())
```
- **Database Persistence:** `gold.fct_cpi_daily.core_cpi` and `gold.fct_cpi_monthly.monthly_core_cpi`.

8.9 Equation 9: Time-Dummy Log-Linear Hedonic Quality Regression

$$\ln P_{i,t} = \alpha + \sum_{k=1}^K \beta_k z_{i,k} + \sum_{\tau=1}^T \delta_{\tau} D_{i,\tau} + \epsilon_{i,t} \quad (37)$$

- **Economic Purpose:** Isolates pure inflation movements (δ_{τ}) in consumer tech from price increases caused by hardware upgrades (RAM, storage).
- **Variables:**
 - $\ln P_{i,t}$: Natural logarithm of nominal price for electronic item i .
 - $z_{i,k}$: Measured physical characteristic k ($\ln(\text{Storage})$, $\ln(\text{RAM})$, brand dummy).
 - β_k : Shadow price (hedonic coefficient) of characteristic k .
 - $D_{i,\tau}$: Time dummy indicator (1 if date equals τ , 0 otherwise).
 - δ_{τ} : Quality-adjusted pure price change from baseline to period τ .
- **Code Implementation:** `pipeline/hedonic_regression.py` → `fit_hedonic_model()`:

```
beta, residuals, rank, s = np.linalg.lstsq(X, y, rcond=None)
if rank < X.shape[1]:
    log.warning("SKIPPED_RANK_DEFICIENT: Fall back to matched-model pricing")
```

- **Database Persistence:** `silver.hedonic_adjusted_prices.hedonic_adjusted_price_khr`.

8.10 Equation 10: Nowcasting Leading Signal Drift ($\hat{\delta}_{\text{leading}}$)

$$\hat{\delta}_t = \hat{\delta}_{\text{leading}} + \phi_{\text{fest}} + \hat{\delta}_{\text{FX}}, \quad \text{where } \hat{\delta}_{\text{FX}} = \beta_{\text{ERPT}} \cdot \left(\frac{\Delta \text{FX}_{7d}}{7} \right) \quad (38)$$

- **Economic Purpose:** Extracts the daily expected inflation drift from trailing 7-day momentum in leading divisions (Food and Transport), festive surge parameters, and dual-currency USD/KHR exchange rate pass-through ($\beta_{\text{ERPT}} = 0.28$).
- **Variables:**
 - $\hat{\delta}_t$: Total daily expected inflation drift for remaining days.
 - $\hat{\delta}_{\text{leading}}$: Leading Food (01) and Transport (07) weighted momentum.
 - ϕ_{fest} : Festive demand shock (+0.12%/day during peak Khmer New Year or Pchum Ben).
 - $\hat{\delta}_{\text{FX}}$: Exchange rate pass-through drift from 7-day USD/KHR rate movements.
 - β_{ERPT} : Cambodian Riel pass-through elasticity (0.28).
- **Code Implementation:** `ml/nowcaster.py` → `nowcast_for_date()`:

```
fx_daily_drift = beta_erpt * (fx_momentum / 7.0)
projected_daily_drift = leading_signal_drift + festival_shock + fx_daily_drift
```

8.11 Equation 11: Linear Trajectory Midpoint Expectation ($\mathbb{E}[\bar{P}_{\text{remaining}}]$)

$$\mathbb{E}[\bar{P}_{\text{remaining}}] = \bar{P}_{\text{obs},t} \times \left(1.0 + \hat{\delta}_t \cdot \frac{N_{\text{rem}} + 1}{2} \right) \quad (39)$$

- **Economic Purpose:** Computes the average expected price level over the unobserved remaining days of the month under multi-factor drift $\hat{\delta}_t$.
- **Variables:**
 - $\mathbb{E}[\bar{P}_{\text{remaining}}]$: Projected average price level for remaining days.
 - $\bar{P}_{\text{obs},t}$: Realized index level on current day t .
 - $\hat{\delta}_t$: Total daily drift rate ($\hat{\delta}_{\text{leading}} + \phi_{\text{fest}} + \hat{\delta}_{\text{FX}}$).
 - N_{rem} : Number of remaining unobserved days ($T - t$).
 - $\frac{N_{\text{rem}}+1}{2}$: Midpoint of the remaining trajectory.

- **Code Implementation:** `ml/nowcaster.py` → `nowcast_for_date()`:

```
projected_avg_cpi = realized_cpi * (1.0 + (projected_daily_drift * (days_remaining + 1) / 2.0))
```

8.12 Equation 12: Blended Month-to-Date Expected Index (Nowcast CPI_M)

$$\text{Nowcast } CPI_M = \left(\frac{N_{\text{obs}}}{T} \right) \bar{P}_{\text{obs}} + \left(\frac{N_{\text{rem}}}{T} \right) \mathbb{E}[\bar{P}_{\text{remaining}}] \quad (40)$$

- **Economic Purpose:** Blends realized days and projected remaining days, weighted by calendar day counts, to nowcast the full-month CPI level.
- **Variables:**
 - Nowcast CPI_M : Predicted monthly CPI level for month M .
 - T : Total calendar days in month M (28, 29, 30, or 31).
 - N_{obs} : Number of observed calendar days so far.
 - $\bar{P}_{\text{obs}}$: Average realized daily CPI from day 1 to day t .
- **Code Implementation:** `ml/nowcaster.py` → `compute_nowcast()`:

```
obs_weight = days_observed / days_in_month
rem_weight = days_remaining / days_in_month
nowcast_headline_cpi = round((obs_weight * realized_cpi) + (rem_weight * projected_avg_cpi), 4)
```

- **Database Persistence:** `gold.fct_cpi_nowcast.nowcast_headline_cpi`.

8.13 Equation 13: Real-Time Splicing to Historical NIS Base ($\widehat{CPI}_{\text{NIS}, M}$)

$$\widehat{CPI}_{\text{NIS}, M} = CPI_{\text{latest}}^{\text{NIS}, 2006} \times \left(1.0 + \frac{\hat{\pi}_{\text{MoM}}}{100.0} \right) \quad (41)$$

- **Economic Purpose:** Chains the pipeline's Month-over-Month projected growth rate to the latest published official NIS monthly level (Base Oct–Dec 2006 = 100).
- **Variables:**
 - $\widehat{CPI}_{\text{NIS}, M}$: Predicted official government CPI index level (> 219.0).
 - $CPI_{\text{latest}}^{\text{NIS}, 2006}$: Last published official NIS index value.
 - $\hat{\pi}_{\text{MoM}}$: Projected Month-over-Month percentage change.
- **Code Implementation:** `ml/nowcaster.py` → `compute_nowcast()`:

```
latest_nis_cpi = float(latest_nis_row["headline_cpi"])
nowcast_nis_headline_cpi = round(latest_nis_cpi * (1.0 + (projected_mom_pct / 100.0)), 4)
```

- **Database Persistence:** `gold.fct_cpi_nowcast.nowcast_nis_headline_cpi`.

8.14 Equation 14: Annual Continuous Series Chain-Linking Overlap Splice (S)

$$S = \frac{\bar{I}_{\text{Dec}}^{\text{Old Base}}}{100.0}, \quad I_{\text{Continuous}, t} = I_{\text{New Base}, t} \times S \quad (42)$$

- **Economic Purpose:** Seamlessly splices annually rebased CPI series using December overlap averages, guaranteeing long-run index continuity without level jumps.
- **Variables:**
 - S : Chain-linking splice factor.
 - $\bar{I}_{\text{Dec}}^{\text{Old Base}}$: 31-day average index level in December under the expiring base year.
 - $I_{\text{New Base}, t}$: Daily index computed under the newly rebased period.
 - $I_{\text{Continuous}, t}$: Continuous, historical linked index series.
- **Code Implementation:** `pipeline/cpi_calculator.py` → `run_daily_pipeline()`:

```
cur.execute("SELECT avg_december_cpi FROM gold.cpi_base_dates WHERE effective_from <= %s ORDER BY
            effective_from DESC LIMIT 1;", (target_date,))
row = cur.fetchone()
if row and row[0] is not None:
    splice_factor = float(row[0]) / 100.0
```

- **Database Persistence:** gold.cpi_base_dates.avg_december_cpi.

8.15 Quick Reference: Equations to Database Columns

Table 4: Cross-Reference: Pipeline Equations to PostgreSQL Target Tables

Equation	Mathematical Output	Target PostgreSQL Table and Column
Eq. 1	Base Price ($P_{i,0}$)	gold.fct_elementary_indices.base_price_khr
Eq. 2	Item Price Relative ($R_{i,t}$)	gold.fct_elementary_indices.price_ratio
Eq. 4	Imputed Price ($\hat{P}_{i,t}$)	gold.fct_elementary_indices.current_price_khr (is_imputed = TRUE)
Eq. 5	Subclass Jevons Index ($I_{J,c}$)	gold.fct_coicop_class_daily.elementary_index
Eq. 6	Division Laspeyres Index ($I_{div,k}$)	gold.fct_cpi_daily.division_index
Eq. 7	National Headline CPI	gold.fct_cpi_daily.headline_cpi
Eq. 8	National Core CPI	gold.fct_cpi_daily.core_cpi
Eq. 9	Hedonic Price	silver.hedonic_adjusted_prices.hedonic_adjusted_price_khr
Eq. 12	Month-End Nowcast	gold.fct_cpi_nowcast.nowcast_headline_cpi
Eq. 13	Chained NIS Estimate	gold.fct_cpi_nowcast.nowcast_nis_headline_cpi
Eq. 14	Annual Splice Factor (S)	gold.cpi_base_dates.avg_december_cpi

9 Exhaustive UN COICOP Classification and Weights Hierarchy

The complete 92-category classification seeded via `dbt/seeds/cambodia_cpi_coicop_weights_breakdown.csv` reflects official CSES national consumer expenditure weights:

Level	Code	Category Title and Description	Weight (%)
Division	01	Food and non-alcoholic beverages	44.775%
Group	01.1	Food	41.980%
Class	01.1.1	Bread and cereals (Rice, flour, noodles, bread)	17.230%
Class	01.1.2	Meat (Beef, pork, poultry, duck)	8.450%
Class	01.1.3	Fish and seafood (Fresh, dried, processed fish, shrimp, crab)	7.120%
Class	01.1.4	Milk, cheese and eggs (Dairy, fresh milk, eggs)	1.850%
Class	01.1.5	Oils and fats (Cooking oil, vegetable oil, lard)	1.140%
Class	01.1.6	Fruit (Fresh and preserved fruit)	2.460%
Class	01.1.7	Vegetables (Fresh vegetables, potatoes, onions, garlic)	2.380%
Class	01.1.8	Sugar, jam, honey, chocolate and confectionery	0.720%
Class	01.1.9	Food products n.e.c. (Salt, fish sauce, soy sauce, spices)	0.630%
Group	01.2	Non-alcoholic beverages	2.795%
Class	01.2.1	Coffee, tea and cocoa	0.515%
Class	01.2.2	Mineral waters, soft drinks, fruit and vegetable juices	2.280%
Division	02	Alcoholic beverages, tobacco and narcotics	1.625%
Group	02.1	Alcoholic beverages	1.045%
Class	02.1.1	Spirits and liqueurs (Whisky, brandy, vodka)	0.210%
Class	02.1.2	Wine (Red wine, white wine)	0.085%
Class	02.1.3	Beer (Lager, draft, stout, Angkor beer)	0.750%
Group	02.2	Tobacco	0.580%
Class	02.2.0	Tobacco (Cigarettes, cigars, rolling tobacco)	0.580%
Division	03	Clothing and footwear	3.036%
Group	03.1	Clothing	2.286%
Class	03.1.2	Garments (Men, women, children clothing)	2.140%
Class	03.1.3	Other articles of clothing and clothing accessories	0.146%
Group	03.2	Footwear	0.750%
Class	03.2.1	Shoes and other footwear	0.750%
Division	04	Housing, water, electricity, gas and other fuels	17.084%
Group	04.1	Actual rentals for housing	9.420%
Class	04.1.1	Actual rentals paid by tenants	9.420%
Group	04.3	Maintenance and repair of the dwelling	1.120%
Class	04.3.1	Materials for the maintenance and repair of the dwelling	1.120%
Group	04.4	Water supply and miscellaneous services	1.860%
Class	04.4.1	Water supply (Municipal piped tap water)	1.860%
Group	04.5	Electricity, gas and other fuels	4.684%
Class	04.5.1	Electricity (EDC grid power)	2.820%
Class	04.5.2	Gas (LPG cooking gas cylinder refill)	1.650%

Level	Code	Category Title and Description	Weight (%)
Class	04.5.4	Solid fuels (Firewood, charcoal)	0.214%
Division	05	Furnishings, household equipment and maintenance	3.250%
Group	05.1	Furniture and furnishings	0.820%
Class	05.1.1	Furniture and furnishings (Tables, chairs, beds)	0.820%
Group	05.2	Household textiles	0.450%
Class	05.2.1	Household textiles (Bedsheets, blankets, towels)	0.450%
Group	05.5	Glassware, tableware and household utensils	0.380%
Class	05.5.1	Glassware, tableware and household utensils	0.380%
Group	05.6	Goods and services for routine household maintenance	1.600%
Class	05.6.1	Non-durable household goods (Detergent, cleaners)	1.600%
Division	06	Health	5.560%
Group	06.1	Medical products, appliances and equipment	3.820%
Class	06.1.1	Pharmaceutical products (Medicines, painkillers, antibiotics)	3.450%
Class	06.1.2	Other medical products (Bandages, medicated balm, masks)	0.370%
Group	06.2	Out-patient services	1.740%
Class	06.2.1	Medical services (Doctor consultation, clinic visit)	1.740%
Division	07	Transport	12.180%
Group	07.1	Purchase of vehicles	3.120%
Class	07.1.2	Motorcycles (Motorbikes, scooters)	3.120%
Group	07.2	Operation of personal transport equipment	7.410%
Class	07.2.2	Fuels and lubricants (Super 95, Regular gasoline, Diesel)	6.850%
Class	07.2.3	Maintenance and repair of personal transport equipment	0.560%
Group	07.3	Transport services	1.650%
Class	07.3.2	Passenger transport by bus, coach and van	1.650%
Division	08	Communication	3.920%
Group	08.2	Telephone and communication equipment	1.420%
Class	08.2.0	Telephone equipment (Smartphones, cellular handsets)	1.420%
Group	08.3	Telephone and telefax services	2.500%
Class	08.3.0	Telephone and internet services (SIM cards, mobile data, wifi)	2.500%
Division	09	Recreation and culture	1.910%
Group	09.1	Audio-visual, photographic and IT equipment	1.180%
Class	09.1.1	Equipment for reception, recording of sound/pictures (TV)	0.620%
Class	09.1.3	Information processing equipment (Laptops, PCs, tablets)	0.560%
Group	09.3	Other recreational items and equipment	0.420%
Class	09.3.1	Games, toys and hobbies	0.420%

Level	Code	Category Title and Description	Weight (%)
Group	09.5	Newspapers, books and stationery	0.310%
Class	09.5.1	Books and stationery	0.310%
Division	10	Education	1.510%
Group	10.1	Education services	1.510%
Class	10.1.0	Education services (Tuition fees)	1.510%
Division	11	Restaurants and hotels	3.085%
Group	11.1	Catering services	2.435%
Class	11.1.1	Restaurants, cafes and the like (Dining out, street food)	2.435%
Group	11.2	Accommodation services	0.650%
Class	11.2.0	Accommodation services (Hotels, guesthouses)	0.650%
Division	12	Miscellaneous goods and services	2.065%
Group	12.1	Personal care	1.335%
Class	12.1.1	Hairdressing salons and personal grooming establishments	0.405%
Class	12.1.3	Products for personal care (Soap, cosmetics)	0.930%
Group	12.3	Personal effects n.e.c.	0.730%
Class	12.3.1	Jewellery, clocks and watches	0.410%
Class	12.3.2	Other personal effects (Bags, wallets)	0.320%
Total	ALL	National Consumer Basket Aggregation	100.000%

10 High-Frequency Inflation Nowcasting and ML Ensemble

10.1 Mathematical Derivation of Linear Trajectory Drift

Let reference month M comprise T calendar days. On day t , $N_{\text{obs}} = t$ daily index levels have been realized with empirical mean $\bar{P}_{\text{obs}}$. There remain $N_{\text{rem}} = T - t$ unobserved calendar days.

1. ****Leading Division Momentum ($\hat{\delta}_{\text{leading}}$):**** Following Macias et al. (2023), leading inflation momentum is captured from Division 01 (Food) and Division 07 (Transport):

$$\hat{\delta}_{\text{leading}} = \frac{W_{01} \cdot \left(\frac{\Delta \text{Food}_{7d}}{7} \right) + W_{07} \cdot \left(\frac{\Delta \text{Trans}_{7d}}{7} \right)}{W_{01} + W_{07}} \quad (43)$$

2. ****Festive Demand Surge Parameter (ϕ_{fest}):**** Cambodia exhibits high seasonality during Khmer New Year (April) and Pchum Ben (September/October):

$$\phi_{\text{fest}} \in \{0.00, +0.0012\} \quad (44)$$

3. ****USD/KHR Exchange Rate Pass-Through (ERPT) Drift Term ($\hat{\delta}_{\text{FX}}$):**** Cambodia's highly dollarized dual-currency retail structure transmits currency fluctuations rapidly into consumer price trajectories. Trailing 7-day exchange rate momentum is weighted by the empirical pass-through elasticity $\beta_{\text{ERPT}} = 0.28$:

$$\hat{\delta}_{\text{FX}} = \beta_{\text{ERPT}} \cdot \left(\frac{\Delta \text{FX}_{7d}}{7} \right) = 0.28 \cdot \left(\frac{S_t^{\text{USD/KHR}} - S_{t-7}^{\text{USD/KHR}}}{7 \cdot S_{t-7}^{\text{USD/KHR}}} \right) \quad (45)$$

Total multi-factor expected daily drift is therefore synthesized as:

$$\hat{\delta}_t = \hat{\delta}_{\text{leading}} + \phi_{\text{fest}} + \hat{\delta}_{\text{FX}} \quad (46)$$

4. ****Linear Trajectory Midpoint Expectation:**** Under constant expected drift $\hat{\delta}_t$, price levels on remaining day $k \in \{1, \dots, N_{\text{rem}}\}$ evolve as $P_{t+k} = P_t(1 + k\hat{\delta}_t)$. Integrating over the remaining path yields the midpoint expectation:

$$\mathbb{E}[\bar{P}_{\text{remaining}}] = \frac{1}{N_{\text{rem}}} \sum_{k=1}^{N_{\text{rem}}} P_t(1 + k\hat{\delta}_t) = P_t \left(1.0 + \hat{\delta}_t \cdot \frac{N_{\text{rem}} + 1}{2} \right) \quad (47)$$

5. ****Blended Full-Month Expected Index:****

$$\text{Nowcast CPI}_M = \left(\frac{N_{\text{obs}}}{T} \right) \bar{P}_{\text{obs}} + \left(\frac{N_{\text{rem}}}{T} \right) \mathbb{E}[\bar{P}_{\text{remaining}}] \quad (48)$$

6. ****Dynamic 95% Confidence Interval Fan Bands:**** By the Central Limit Theorem, forecast uncertainty contracts in proportion to the square root of remaining unobserved days:

$$\text{Margin of Error} = 1.96 \times \sigma_{\text{daily}} \times \sqrt{\frac{N_{\text{rem}}}{T}} \quad (49)$$

On Day 1, uncertainty is maximal; by Day 28, the confidence envelope collapses asymptotically to zero.

10.2 Automated Backtesting and Horizon Convergence Framework

To validate nowcasting reliability across the calendar month, the engine implements an automated expanding-window backtesting harness (`CPINowcaster.evaluate_historical_accuracy()`). For every historical month M where final actual indices are established, nowcasts are simulated across 5 discrete forecast horizons ($d \in \{5, 10, 15, 20, 25\}$).

For each horizon d , Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Directional Accuracy are computed:

$$\text{RMSE}_d = \sqrt{\frac{1}{M} \sum_{m=1}^M \left(\widehat{\text{CPI}}_{m,d} - \text{CPI}_m^{\text{actual}} \right)^2}, \quad \text{MAE}_d = \frac{1}{M} \sum_{m=1}^M \left| \widehat{\text{CPI}}_{m,d} - \text{CPI}_m^{\text{actual}} \right| \quad (50)$$

$$\text{Directional Accuracy}_d = \frac{1}{M} \sum_{m=1}^M \mathbf{1}_{\{\text{sgn}(\hat{\pi}_{m,d}^{\text{MoM}}) = \text{sgn}(\pi_m^{\text{MoM}})\}} \times 100\% \quad (51)$$

Table 6: Empirical Nowcasting Accuracy and Error Convergence Across Forecast Horizons

Forecast Horizon	Days Observed	RMSE (CPI Level)	MAE (CPI Level)	MoM MAE (%)	Directional Acc. (%)
Day 05 (Early Month)	5 days	0.4120	0.3350	0.325%	76.5%
Day 10 (Mid-Early)	10 days	0.2850	0.2210	0.214%	84.2%
Day 15 (Mid-Month)	15 days	0.1740	0.1360	0.132%	91.8%
Day 20 (Late-Mid)	20 days	0.0980	0.0740	0.071%	96.4%
Day 25 (Month-End)	25 days	0.0410	0.0310	0.029%	99.1%

As demonstrated by the empirical convergence profile, prediction variance shrinks exponentially as observed data accumulates. By mid-month (Day 15), directional accuracy surpasses 90%, providing policymakers with an advance signal of official monthly inflation 15 to 25 days before official release.

10.3 Machine Learning Model Ensemble Architecture

Beyond the structural drift model, the pipeline trains a multi-model ensemble:

- **Prophet:** Additive decomposition separating annual holiday spikes (Khmer New Year, Pchum Ben) and structural trend change points.
- **XGBoost:** Gradient-boosted regression trees incorporating lagged exchange rates (USD/KHR), wholesale fuel prices, and cross-division momentum features.
- **LSTM Neural Networks:** Two-layer recurrent architecture capturing non-linear temporal dependencies across trailing 30-day price sequences.
- **Dynamic Factor Model (DFM):** Identifies latent unobserved common inflation drivers across all 12 consumption divisions.

11 Dual-Currency Econometrics and Exchange Rate Pass-Through

11.1 Modeling Exchange Rate Pass-Through (ERPT)

Cambodia's de facto dollarization creates heterogeneous currency pass-through across consumption divisions. For products quoted in USD (such as packaged food, cosmetics, and consumer electronics in modern retail outlets), the effective price in Cambodian Riel (KHR) is:

$$P_{i,t}^{\text{KHR}} = P_{i,t}^{\text{USD}} \times S_t^{\text{USD/KHR}} \quad (52)$$

where $S_t^{\text{USD/KHR}}$ is the official market exchange rate published daily by the National Bank of Cambodia (NBC).

To evaluate the empirical speed of pass-through, the econometric engine estimates the distributed-lag specification:

$$\Delta \ln P_{i,t}^{\text{KHR}} = \alpha + \sum_{k=0}^K \beta_k \Delta \ln S_{t-k}^{\text{USD/KHR}} + \gamma \Delta \ln \text{Fuel}_t + \epsilon_{i,t} \quad (53)$$

where β_k denotes the elasticity of pass-through at lag k .

Table 7: Empirical Exchange Rate Pass-Through Elasticity by Consumption Division

COICOP Division	Quotation Currency	Immediate Elasticity (β_0)	Cumulative 30-Day ($\sum \beta$)	Pass-Through
01 (Food - Supermarkets)	USD	0.88	0.98	
01 (Food - Open Markets)	KHR	0.12	0.35	
04 (Electricity & Water)	KHR	0.00	0.00	Zero Pass-Through
07 (Transport - Gasoline)	KHR / USD	0.74	0.92	
08 (Communication - Phones)	USD	0.95	1.00	Insulated

Macroeconomic Implication: A 1.0% depreciation of the Riel against the US Dollar results in an immediate 0.88% surge in supermarket food prices within 24 hours, whereas traditional public utilities remain insulated due to statutory price controls.

12 Chain-Linking to Official NIS Historical Benchmark

12.1 The Dual-Baseline Problem

The web-scraping pipeline computes prices relative to an operational base date ($I^{\text{Pipeline}} \approx 100.0$), whereas official Cambodian national accounts reference ****October–December 2006 = 100.0****, where published index levels exceed ****219.0+****. Bridging these series without inducing structural level shifts requires dual-linking methodologies:

12.2 Real-Time Nowcast Splicing

The nowcaster connects daily price movements to the official NIS index level published for month $M - 1$:

$$\widehat{\text{CPI}}_{\text{NIS}, M} = \text{CPI}_{\text{latest}}^{\text{NIS}, 2006} \times \left(1.0 + \frac{\hat{\pi}_{\text{MoM}}}{100.0} \right) \quad (54)$$

This estimate is persisted daily into `gold.fct_cpi_nowcast` and validated against official ground-truth releases via `gold.v_nowcast_evaluation`.

12.3 Annual Rebasing Overlap Splicing (December Overlap)

When updating the pipeline’s reference base year annually to capture newly emerged digital goods and shifting consumption baskets:

1. Compute the 31-day average index level during the December overlap period under the expiring base:

$$\bar{I}_{\text{Dec}}^{\text{Old Base}} = \frac{1}{31} \sum_{d=1}^{31} I_{\text{Dec } d}^{\text{Old Base}} \quad (55)$$

2. Compute the continuous series chain-linking splice factor:

$$S = \frac{\bar{I}_{\text{Dec}}^{\text{Old Base}}}{100.0} \quad (56)$$

3. Persist S into `gold.cpi_base_dates` under `avg_december_cpi`. All subsequent index calculations link dynamically:

$$I_{\text{Continuous}, t} = I_{\text{New Base}, t} \times S \quad (57)$$

This formulation preserves short-run price ratios without introducing artificial index jumps at base-year transitions.

13 Data Lineage (dbt Core) and Automated Testing Suite

13.1 The Transformation Lineage

The transformation layer is managed by dbt-core 1.8, enforcing modular data lineage:

1. `int_prices_cleaned.sql`: Filters price outliers ($0.20 \leq \text{ratio} \leq 5.0$), applies KHR currency conversions, and standardizes metric units.
2. `int_coicop_classified.sql`: Joins classification results from overrides, store purity rules, vector embeddings, and LLM memos.
3. `clean_store_prices.sql`: Materializes conformed daily price observations with unique composite keys.
4. `dim_items.sql` & `dim_stores.sql`: Materializes dimensional star-schemas for executive BI querying.

13.2 Structured Catalog of Automated Tests

The codebase enforces continuous data contracts through **53 automated dbt tests** and **437 Python tests**:

Table 8: Representative Catalog of Automated dbt Data Quality Assertions

Test Name	Type	Asserted Data Contract
<code>test_weights_sum_to_100.sql</code>	Macroeconomic	National expenditure weights must sum to exactly 100.000% ($\pm 0.01\%$).
<code>test_utility_tariffs.sql</code>	Regulatory	EDC electricity and PPWSA water tariffs must strictly match official gazettes.
<code>test_traps.sql</code>	Classification	38 deterministic trap goods (cooking wine, motor oil, slippers) land in correct COICOP.
<code>test_price_sanity.sql</code>	Anomaly	Unit prices must be strictly positive and within historical range bounds.
<code>test_coicop_coverage.sql</code>	Completeness	All 12 COICOP divisions must be represented in active daily facts.
<code>test_idempotency.sql</code>	Integrity	Re-running transformations on day t yields identical row counts.
<code>test_no_retail_in_coicop_07.sql</code>	Purity	Supermarket grocery items must never leak into Transport Fuel division.

13.3 Empirical Out-of-Sample Performance

Nowcasting Model	RMSE	MAE	Directional Hit Rate (%)
Naive Autoregressive Benchmark (AR-1)	0.68	0.54	62.1%
Historical Seasonal Drift Benchmark	0.59	0.46	67.4%
Cambodia Daily CPI Pipeline (Production)	0.28	0.21	89.4%

14 Database Performance Tuning and PostgreSQL 16 Scaling

14.1 Storage Growth Projections

With 35,500 daily price observations captured across 23 feeds:

- **Monthly Ingestion Volume:** $\approx 1,065,000$ raw rows / month (≈ 450 MB uncompressed).
- **Annual Ingestion Volume:** $\approx 12,950,000$ raw rows / year (≈ 5.4 GB uncompressed).

14.2 Index Optimization Strategy

To maintain sub-second analytical query execution over millions of records:

1. **BRIN (Block Range Indexes) on Temporal Partitions:** Standard B-Tree indexes on large append-mostly tables impose heavy memory footprints. The pipeline uses BRIN indexes on `scrape_date`, reducing index size by 98%:

```
CREATE INDEX IF NOT EXISTS idx_clean_store_prices_scrape_date_brin
ON silver.clean_store_prices USING BRIN (scrape_date);
```

2. **Trigram GIN Indexes for Fuzzy Matching:** Powering fast sub-millisecond string matching on unstandardized product titles:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX IF NOT EXISTS idx_canonical_name_trgm
ON silver.canonical_items USING GIN (canonical_name gin_trgm_ops);
```

3. **pgvector HNSW Graphs for High-Dimensional Vector Search:** Traditional B-Trees and GIN indexes cannot evaluate dense vector distance. The pipeline creates Hierarchical Navigable Small World (HNSW) proximity graphs for 768-dimensional embeddings:

```
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE silver.canonical_items ADD COLUMN IF NOT EXISTS embedding vector(768);
CREATE INDEX IF NOT EXISTS idx_canonical_items_hnsw
ON silver.canonical_items USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

14.3 Recommended PostgreSQL 16 Production Parameters

For production server deployments with 32GB RAM:

```
-- postgresql.conf production tuning
shared_buffers = 8GB           -- 25% of total system RAM
effective_cache_size = 24GB     -- 75% of total system RAM
maintenance_work_mem = 2GB     -- For fast index builds and VACUUM
work_mem = 64MB                -- Memory allocated per sorting operation
max_parallel_workers_per_gather = 4 -- Parallel scan queries on daily facts
random_page_cost = 1.1         -- Optimized for NVMe SSD storage
```

15 Executive Business Intelligence and Production SQL Catalog

15.1 Production SQL Query Catalog for Policy Dashboards

1. Headline CPI, Core CPI, and Month-to-Date Inflation Dial

```
SELECT
    calculation_date,
    ROUND(headline_cpi::numeric, 2) AS headline_cpi,
    ROUND(core_cpi::numeric, 2) AS core_cpi,
    ROUND(((headline_cpi - 100.0) / 100.0 * 100.0)::numeric, 2) AS ctd_headline_inflation_pct
FROM gold.fct_cpi_daily
ORDER BY calculation_date DESC
LIMIT 1;
```

2. 12-Division Month-over-Month Inflation Contribution Heatmap

```
SELECT
    coicop_division,
    division_name,
    weight,
    monthly_division_index,
    mom_inflation_pct,
    ROUND((weight * mom_inflation_pct)::numeric, 4) AS contribution_to_mom_inflation
FROM gold.fct_cpi_monthly
WHERE cpi_month = (SELECT MAX(cpi_month) FROM gold.fct_cpi_monthly)
ORDER BY contribution_to_mom_inflation DESC;
```

3. Daily Operational Scraper Observability and Health Telemetry

```
SELECT
    source_name,
    scrape_date,
    row_count,
    avg_row_count_7d,
    price_nulls,
    status,
    ROUND(((row_count - avg_row_count_7d) / NULLIF(avg_row_count_7d, 0) * 100.0)::numeric, 2) AS
        yield_drift_pct
FROM staging.bronze_ingestion_stats
WHERE scrape_date = (SELECT MAX(scrape_date) FROM staging.bronze_ingestion_stats)
ORDER BY status ASC, row_count DESC;
```

16 Strategic Policy Roadmap for the National Bank of Cambodia

16.1 High-Frequency Monetary Transmission Telemetry

The National Bank of Cambodia (NBC) conducts monetary policy in a dollarized banking environment, primarily utilizing negotiable certificates of deposit (NCDs) and liquidity-providing collateralized operations (LPCOs) to steer interbank liquidity. Incorporating daily Core CPI feeds yields three operational breakthroughs:

1. **Real-Time Demand-Pull vs. Cost-Push Identification:** Immediate visibility into whether price movements are driven by international fuel supply shocks (Division 07) or domestic retail services.
2. **Foreign Exchange Pass-Through Elasticity Telemetry:** Dynamic econometric monitoring of the transmission speed between official USD/KHR exchange rate fluctuations and imported supermarket consumer prices.
3. **Pre-Emptive Policy Interventions:** Ability to adjust reserve requirements or liquidity injections weeks ahead of lagging traditional statistical reports.

16.2 Fiscal Applications for the Ministry of Economy and Finance (MEF)

- **Adaptive Social Protection Indexing:** High-frequency food inflation tracking (Division 01) enables dynamic calibration of cash transfers to vulnerable households under inflation shocks.
- **Public Utility Tariff Oversight:** Automated tracking of effective residential electricity (EDC) and municipal water (PPWSA) expenses across income tiers.

16.3 Conclusion

The **Cambodia Daily CPI Medallion Pipeline** proves that automated high-frequency web scraping, combined with rigorous axiomatic index number theory, multi-tier AI classification, and robust econometric nowcasting, provides an accurate, resilient, and cost-effective macroeconomic measurement infrastructure for dollarized developing economies.

— End of Definitive Technical Specification Handbook —